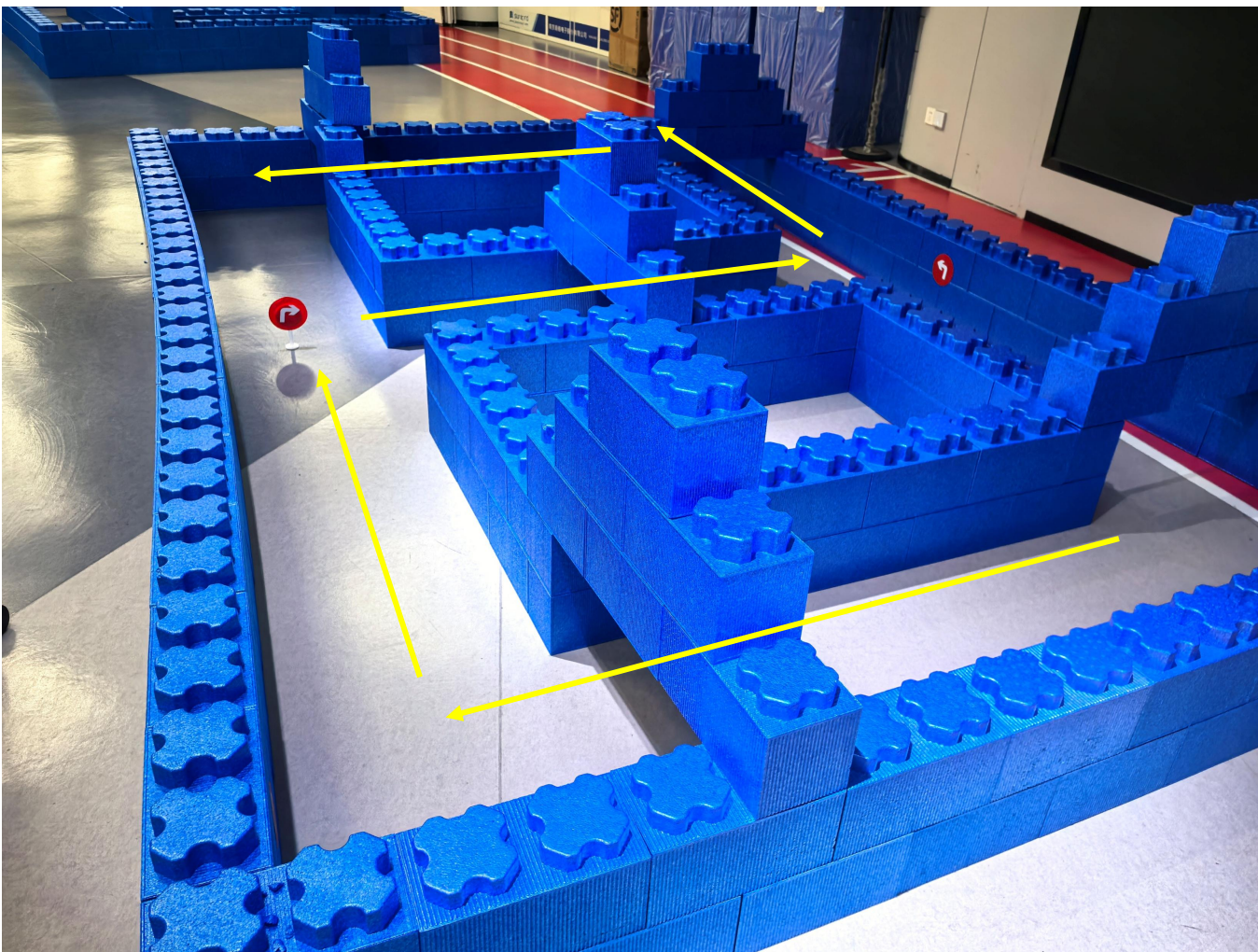




第二部分 视觉信息处理



基于左图8字跑道走S形路线

- 控制小车沿左图中黄色路线行驶,
- 使用车载单目摄像头对路线中的红色标志牌进行识别, 完成相应的转向和停止行动; 若小车能够顺利到达终点, 则视为**任务完成**。

在这一任务中将学习以下内容

1. 树莓派的使用
2. 相机标定与数字图像基础知识
3. 基于PID的车辆巡线控制
4. 基于神经网络的标志牌识别



1

树莓派的使用

2

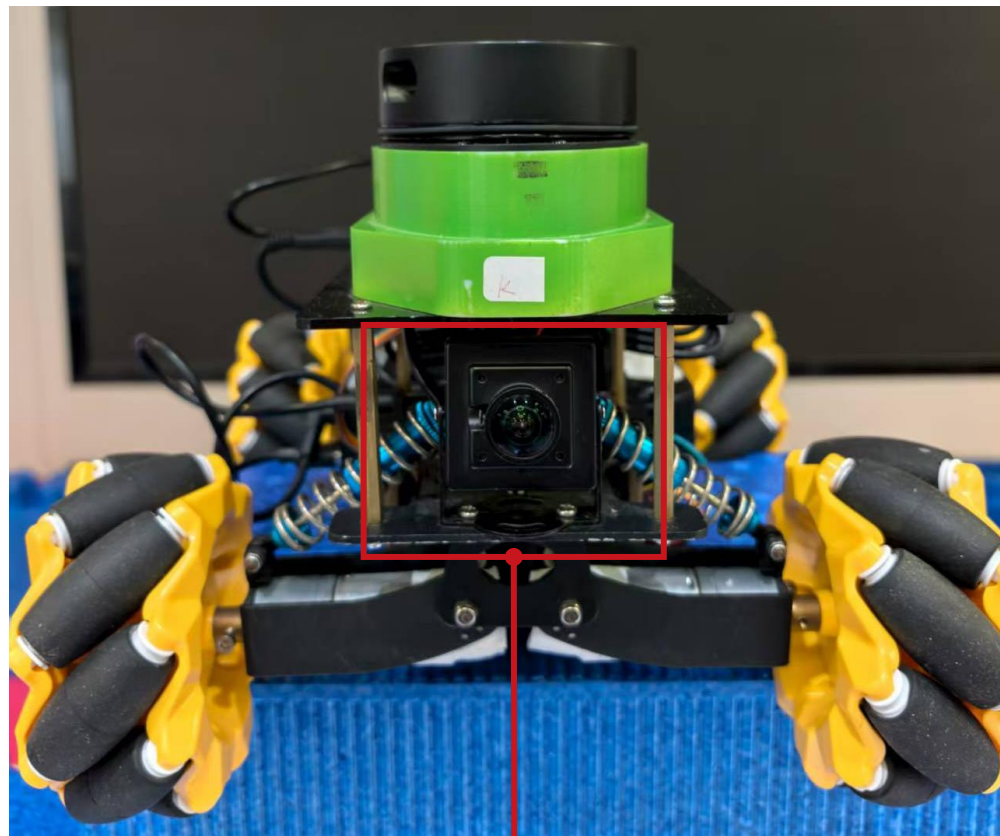
相机标定与数字图像基础知识

3

基于PID的车辆巡线控制

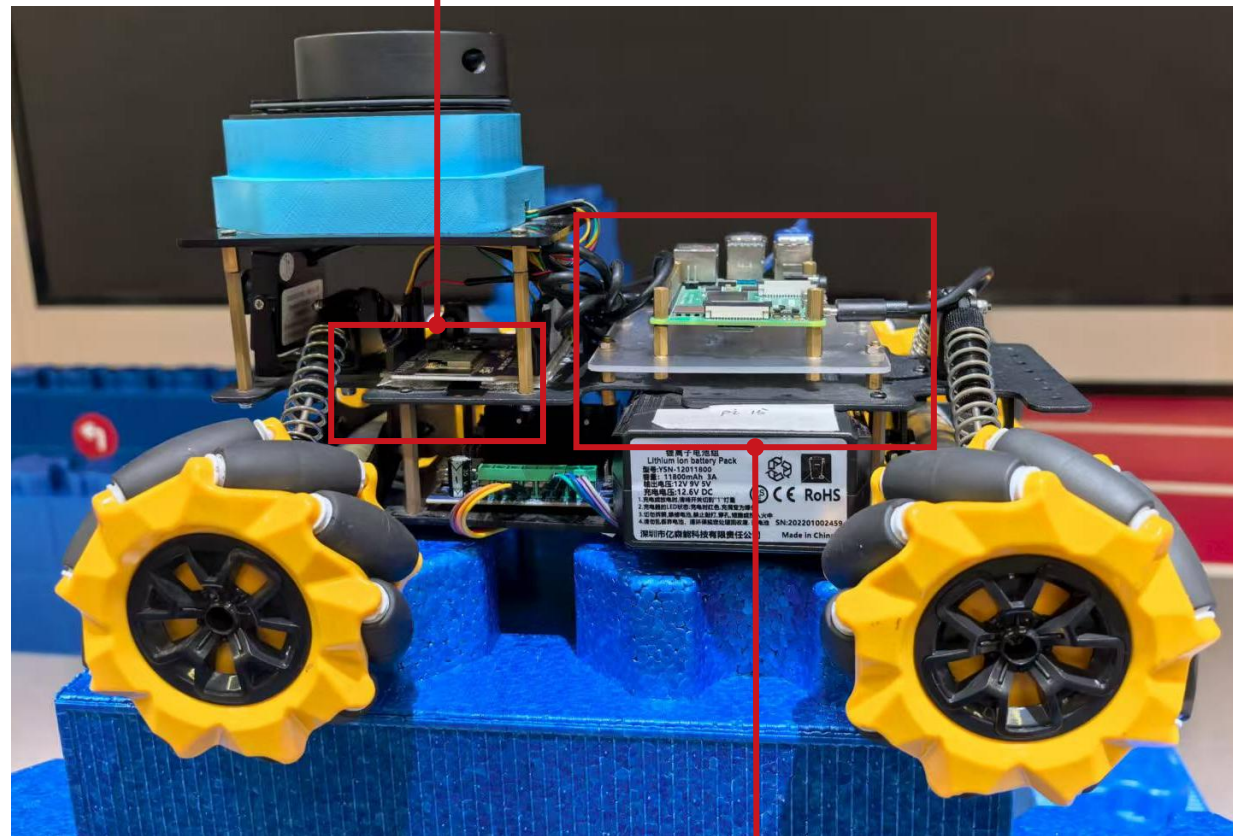
4

基于卷积网络的标志牌识别



前端摄像机

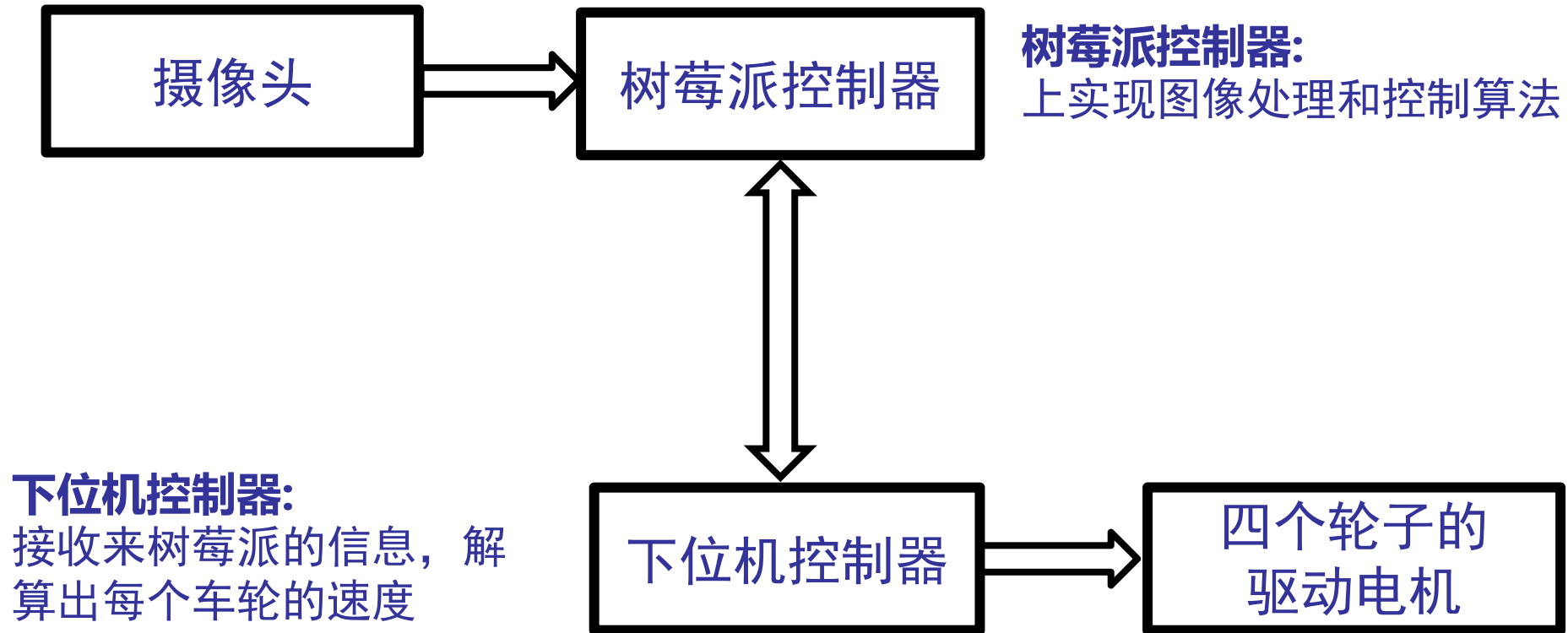
SD卡一张（内有镜像）



树莓派处理器



智能车部件与控制逻辑





树莓派的远程连接

将sd卡接入读卡器：

1. 新建文件 文件名为**ssh** 和 **wpa_supplicant.conf**
2. 可以新建一个ssh.txt 然后重命名**删除.txt** 即可
3. ssh文件是**空的**
4. wpa_supplicant.conf中写入：

```
country=cn
```

```
update_config=1
```

```
ctrl_interface=/var/run/wpa_supplicant
```

```
network={
```

```
ssid="wifi名字"      ## 例如可写成：“pi7” ##
```

```
psk="wifi密码"       ## 例如可写成：“1234567890” ##
```

```
}
```

注意：不能有中文存在。



树莓派的远程连接

步骤1. 打开电脑移动热点，将**热点名称和密码**修改为车子上标注值或在前面程序中设定好的

- 在电脑设置中打开“网络和Internet”
- “移动热点”开关打开

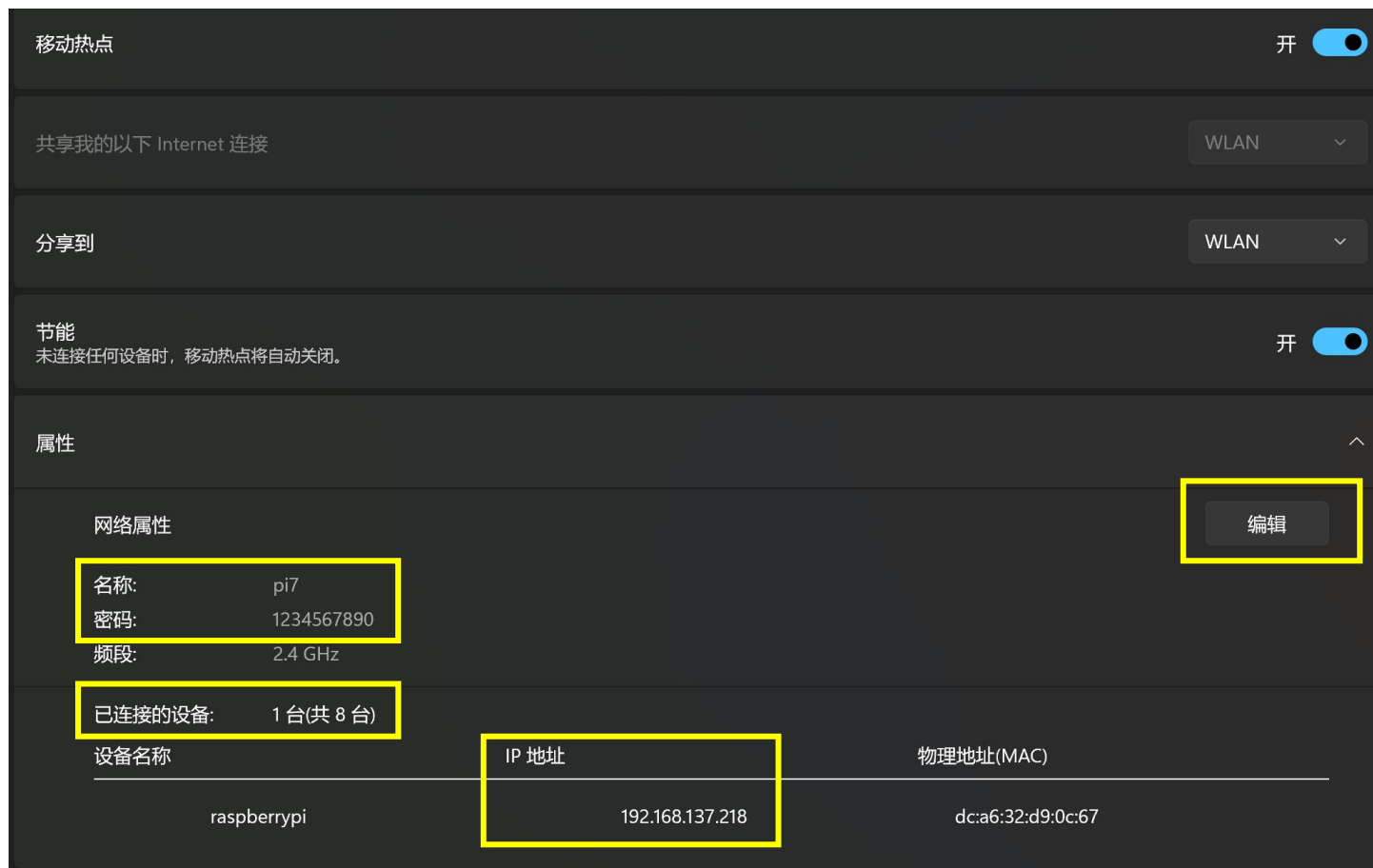




树莓派的远程连接

步骤1. 打开电脑移动热点，将**热点名称**和**密码**修改为车子上标注值或在前面程序中设定好的

- “**移动热点**”开关打开，打开“**编辑**”，输入**名称与密码**；稍等显示“**已连接的设备 1台**”
- 获取小车的**IP地址**，例如下图中的“192.168.137.218”，保存备用





树莓派的远程连接

步骤2. 下载并安装VNC Viewer;

步骤3. 打开VNC view, 显示图形界面:

(1) 输入IP地址, 连接

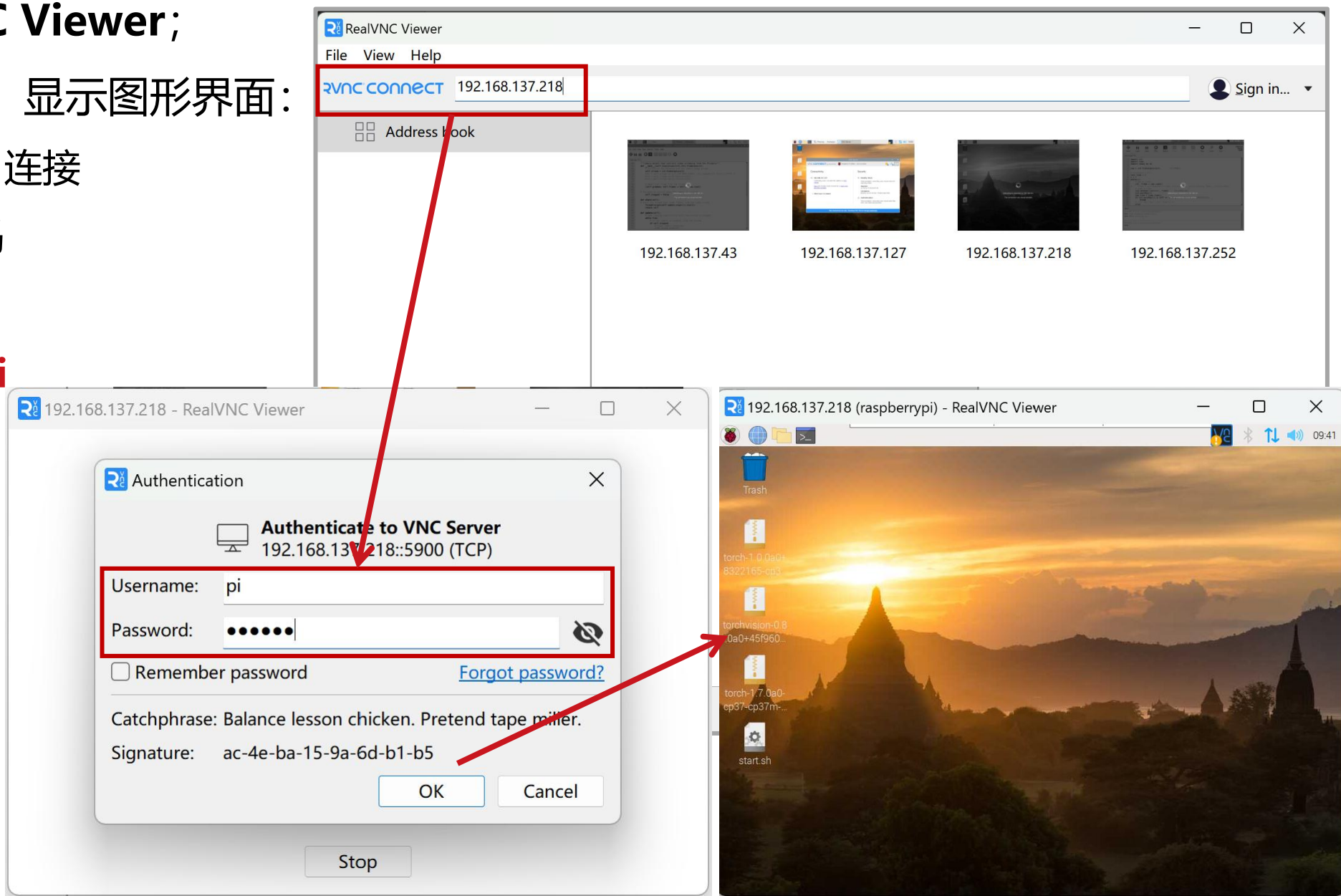
(2) 输入账号密码

账号: **pi**

密码: **cvprpi**

(3) 连接显示

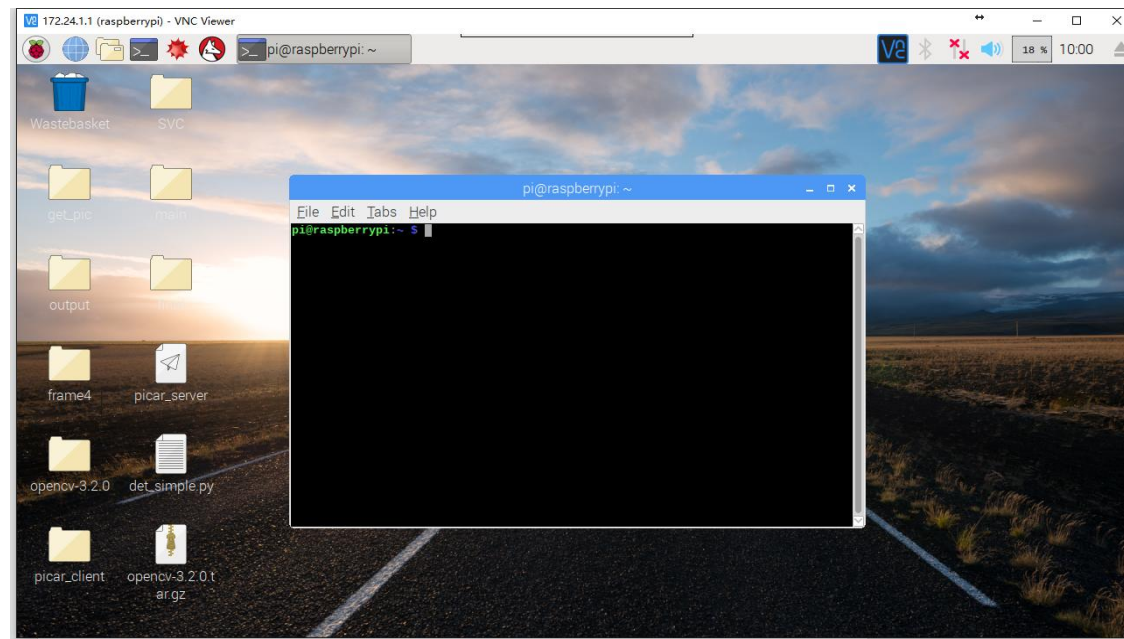
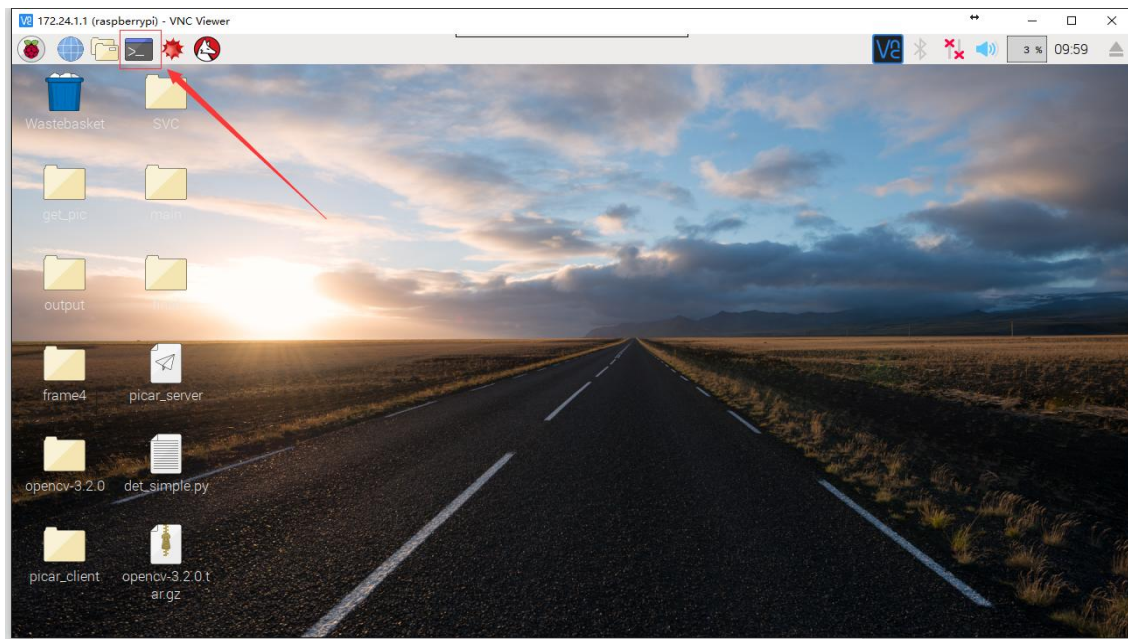
如图即**成功连接**





使用命令行来运行代码：

1. 开启终端



2. 切换到工作目录，运行python代码：

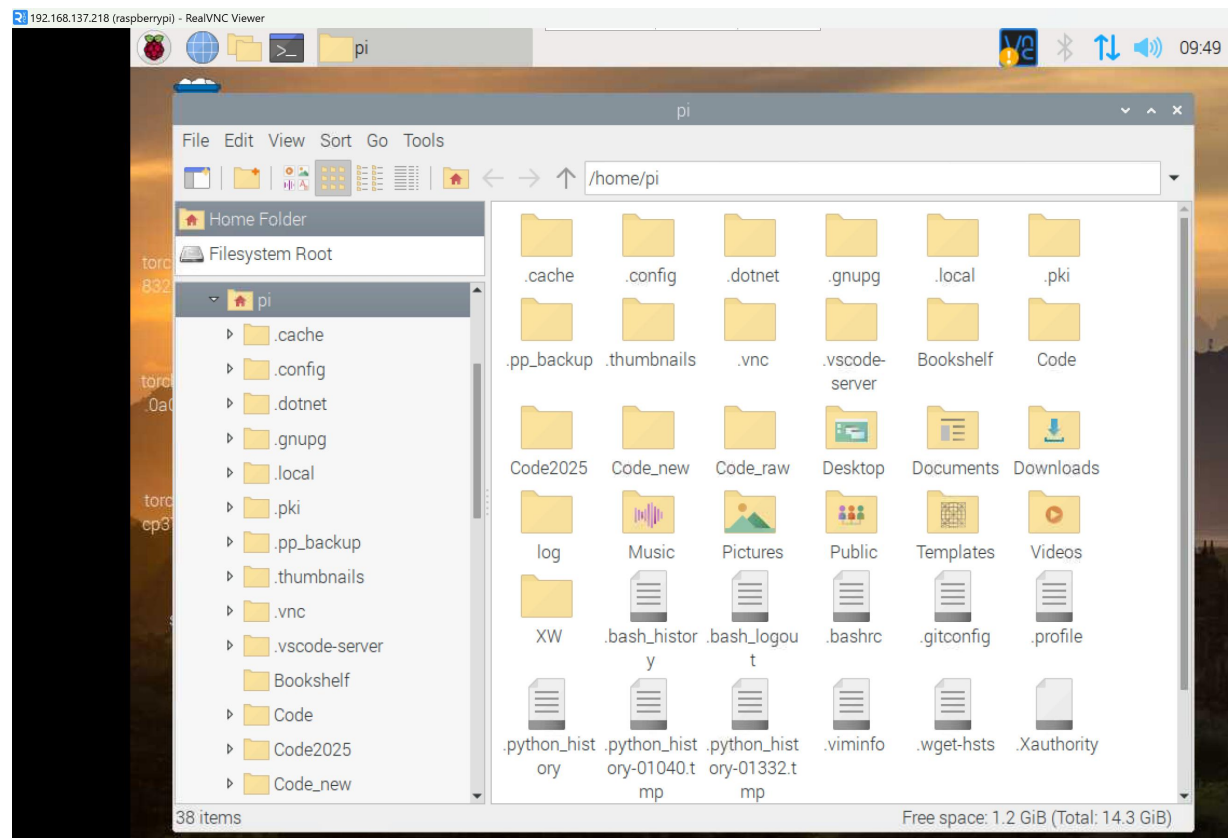
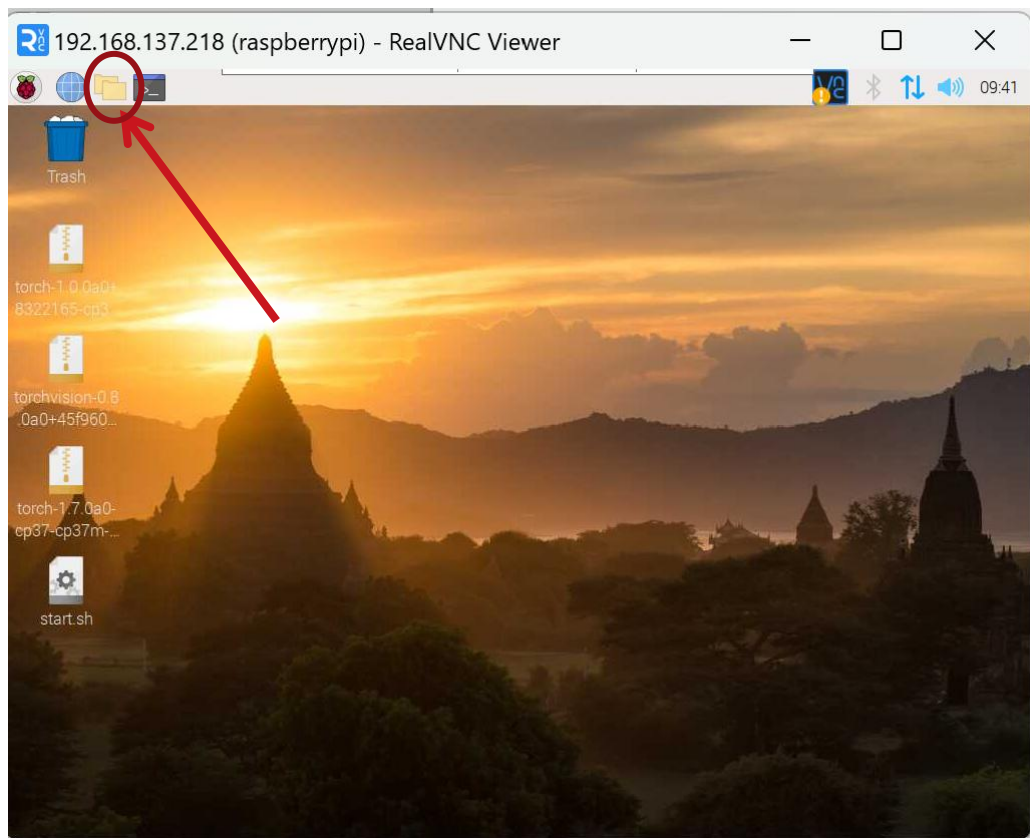


```
cd XXX #修改为想要进入的目录  
python3 xxx.py
```



使用图形化操作界面：

1. 开启终端



2. 进入到工作目录，双击python代码文件



1

树莓派的使用

2

相机标定与数字图像基础知识

3

基于PID的车辆巡线控制

4

基于卷积网络的标志牌识别



由于镜头特性，光学成像经常存在明显的几何畸变。最常见的**畸变类型**可分为两类：

1. 径向畸变 (Radial Distortion)

- 表现为图像像素点沿半径方向发生的位移
- 特点是畸变量随离光心距离的增加而增大

2. 切向畸变 (Tangential Distortion)

- 主要由镜头与成像平面非平行安装引起
- 表现为图像像素点的切向位移
- 会导致矩形物体产生梯形失真现象

因此需要通过**相机标定**建立准确的畸变校正模型。





步骤1.保存摄像头截图

在树莓派上，依次执行以下代码：



```
cd Calibration  
python3 picture.py
```

- 拍摄用于标定的棋盘格10~20张（各个角度/距离）
- 按P键保存摄像头当前图像，按Q键退出当前程序；
- 图像将保存在Calibration/pic目录下

步骤2.根据保存的图片计算矫正参数

继续执行



```
python3 CalibrateCamera.py
```

- 相应的标定参数将保存为Calibration.npz文件
- 后续参数将在运行时加载





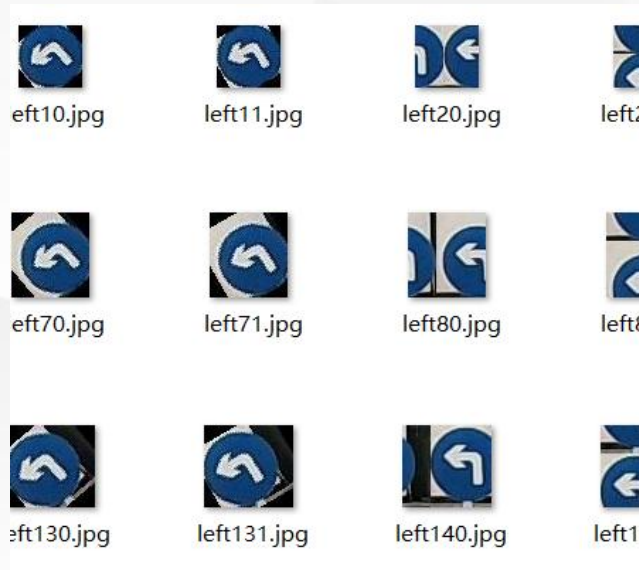
标志牌数据收集和训练

步骤1.采集标志牌图片

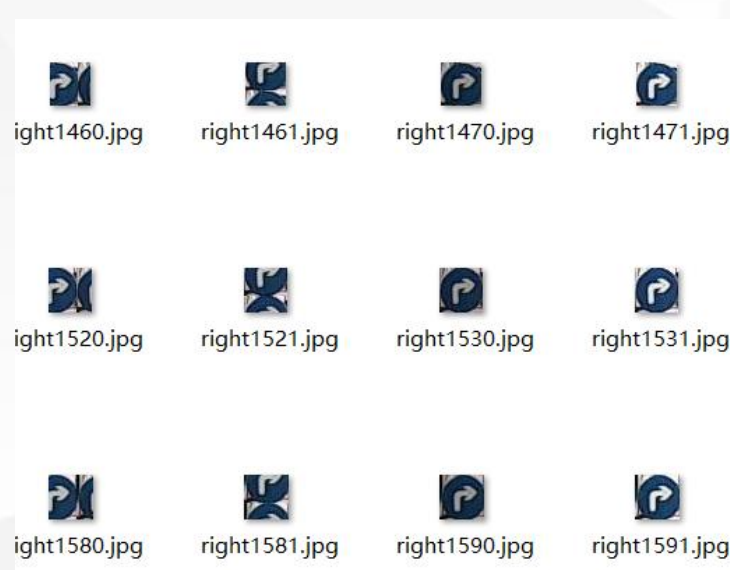
可以继续使用相机标定时时的picture.py文件直接获取图片（需要从各距离和角度对每个标志牌拍摄上百张），也可以使用until文件夹中的video.py文件拍摄一段视频，然后对其进行抽帧、裁剪以获得标志牌不同光照、不同大小、不同背景下的数据。

此外还可以对原始收集到的数据进行仿射变换来达到数据增广的目的，确保网络可以学习到更加鲁棒的特征

左标志牌



右标志牌



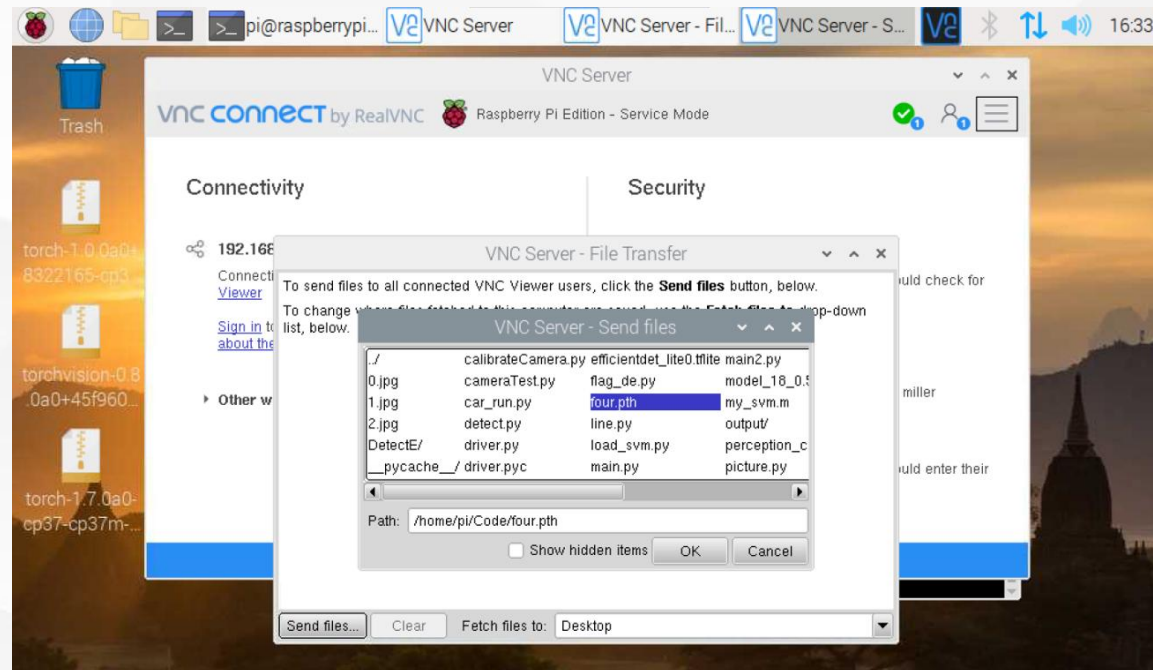
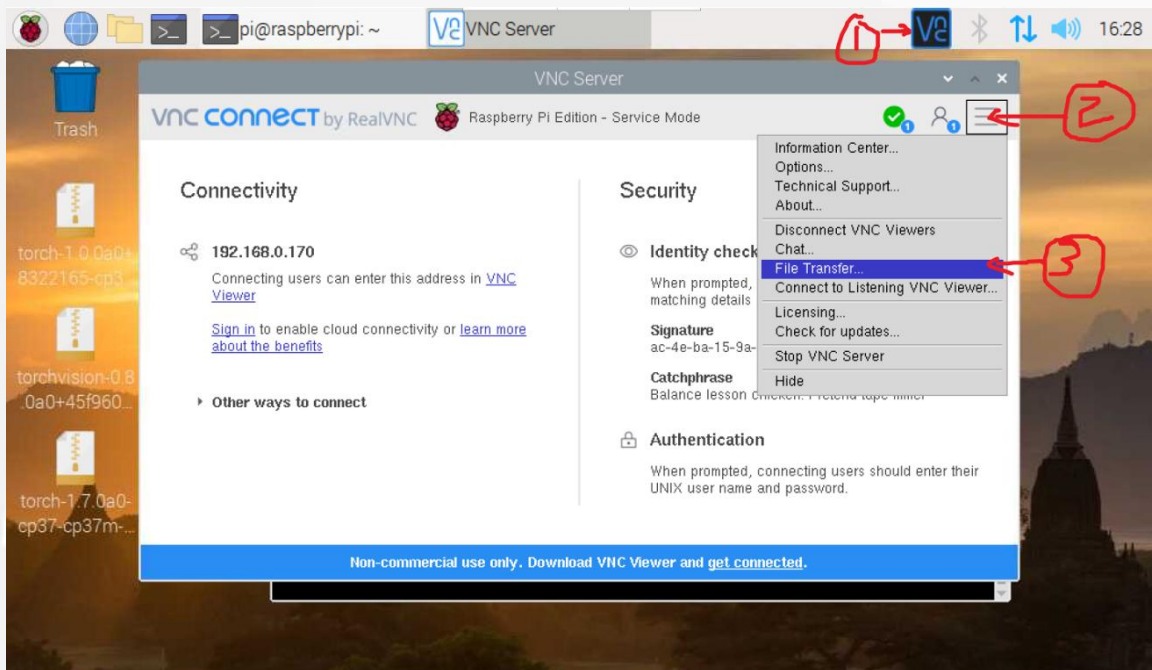


标志牌数据收集和训练

步骤2 .将图片或视频从树莓派传到电脑

使用vnc自带的功能即可，首先选择File Transfer功能

然后选择要传递的文件，点击send files即可传递文件到本地电脑上





步骤3 .在电脑上完成训练

将数据集准备好后，在电脑上运行train_3.py文件就可以训练模型了（该文件由老师提供）。

为使该代码顺利运行，我们需要把数据结构分割成训练集和验证集，每个集中包含4个子文件夹，每个子文件夹分别存放左、右、直行、停止的图片。

```
train_data=datasets.ImageFolder(root='./raw_data/train',transform=data_transform)
test_data=datasets.ImageFolder(root='./raw_data/val',transform=data_transform)
```

我们也可以修改网络结构尝试获取更好的性能，如右图所示。但要注意将树莓派上的net.py文件中的网络结构也修改成一致的。

代码运行结束后会保存一个模型权重参数的文件

```
save_path = "./four_4.pth"
```

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1),
            nn.Conv2d(16, 16, 5),
            nn.ReLU(),
            nn.MaxPool2d(2, stride=2),
            nn.Dropout(0.3),
            nn.Conv2d(16, 32, 5),
            nn.ReLU(),
            nn.MaxPool2d(2, stride=2),
            nn.Dropout(0.3)
        )
        self.fc = nn.Sequential(
            nn.Linear(32*4*4, 100),
            nn.ReLU(),
            nn.Linear(100, 4)
        )

    def forward(self, x):
        x = self.conv(x)
        x = x.view(-1, 32*4*4)
        x = self.fc(x)
        # x = nn.functional.normalize(x)
        return x
```




步骤4 .将训练好的权重传回树莓派

使用scp命令来进行文件的传输

打开终端（windows下可以选择powershell）

输入命令行：

按照提示输入yes和密码（cvprpi）后即可完成传输（输入密码不会显示，在屏幕上是没有变化的）

最后在树莓派上把four_4.pth放到main文件夹（控制小车的主程序和网络结构文件net.py都应该在这里），即可使用神经网络进行标志牌识别。

指令格式： `scp` [选项] 源文件地址 目标地址

例如，现在我的本地电脑桌面下有一个123.txt文件，需要将这个文件传输到树莓派的Code文件夹下

其中树莓派的ip地址为192.168.0.170

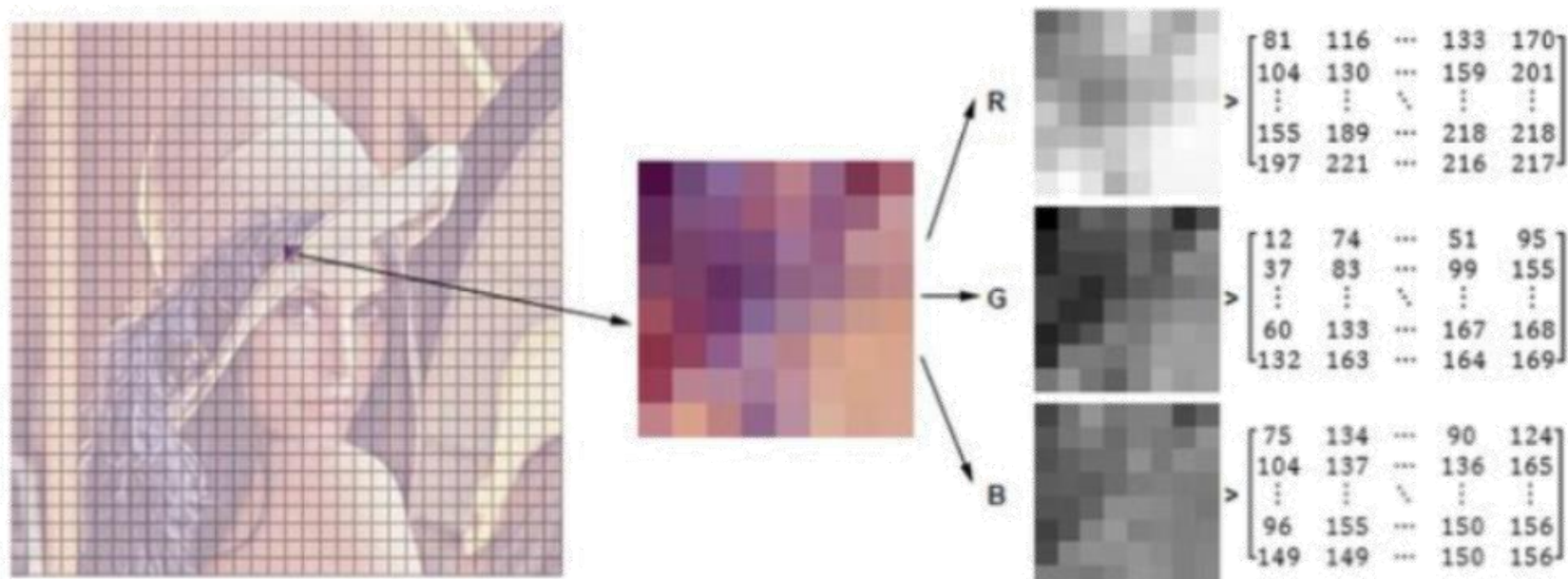
`scp C:\Users\hhf\Desktop\123.txt pi@192.168.0.170:/home/pi/Code`

```
PS C:\Users\hhf\Desktop> scp C:\Users\hhf\Desktop\123.txt pi@192.168.0.170:/home/pi/Code
The authenticity of host '192.168.0.170 (192.168.0.170)' can't be established.
ED25519 key fingerprint is SHA256:DJHh/yqLGqyUmPdKnHMqDvaNZfvb4MwXeGmJdskTQGI.
This host key is known by the following names/addresses:
  C:\Users\hhf/.ssh/known_hosts:31: 192.168.212.163
  C:\Users\hhf/.ssh/known_hosts:34: 192.168.136.163
Are you sure you want to continue connecting (yes/no/[fingerprint])?
Warning: Permanently added '192.168.0.170' (ED25519) to the list of known hosts.
pi@192.168.0.170's password:
123.txt
100% 0 0.0KB/s 00:00
PS C:\Users\hhf\Desktop> |
```



数字图像基础知识

- **数字图像**由连续的模拟图像采样和量化而得。
- 每张图片都是由很多个色点组成的，每个色点称之为一个**像素(Pixel)**。
- 总像素量直接反映图像的采样密度，计算公式为：**分辨率=水平像素数×垂直像素数**。
更高像素密度意味着更精细的采样间隔，在图像放大时能保持更好的锐度和细节表现；
- 像素的值代表图像在该位置的亮度（或灰度），称为图像的**灰度值**。
数字图像像素具有整数坐标和整数灰度值。





三种数字图像

灰度图像是每个像素只有灰度值的图像，因此只有一个通道。如下图，灰度图像每个像素的灰度值是0~255之间的整数，其中0代表黑，255代表白。



黑白图像也叫二值图像。黑白图像同灰度图像，仅有一个通道，与灰度图像不同的是，黑白图像中每个像素仅能取0或者255，分别代表纯黑和纯白。



彩色图像每个像素通常是由红（R）、绿（G）、蓝（B）三个分量来表示的，每个分量介于0~255之间。





- 图像二值化是将一幅灰度图像转换为二值图像的过程，其中每个像素只有两个可能的取值：黑色或白色（或者0和255）。
- 在进行图像二值化时，需要确定一个阈值，将像素值与阈值进行比较，并根据比较结果将像素设置为黑色或白色。

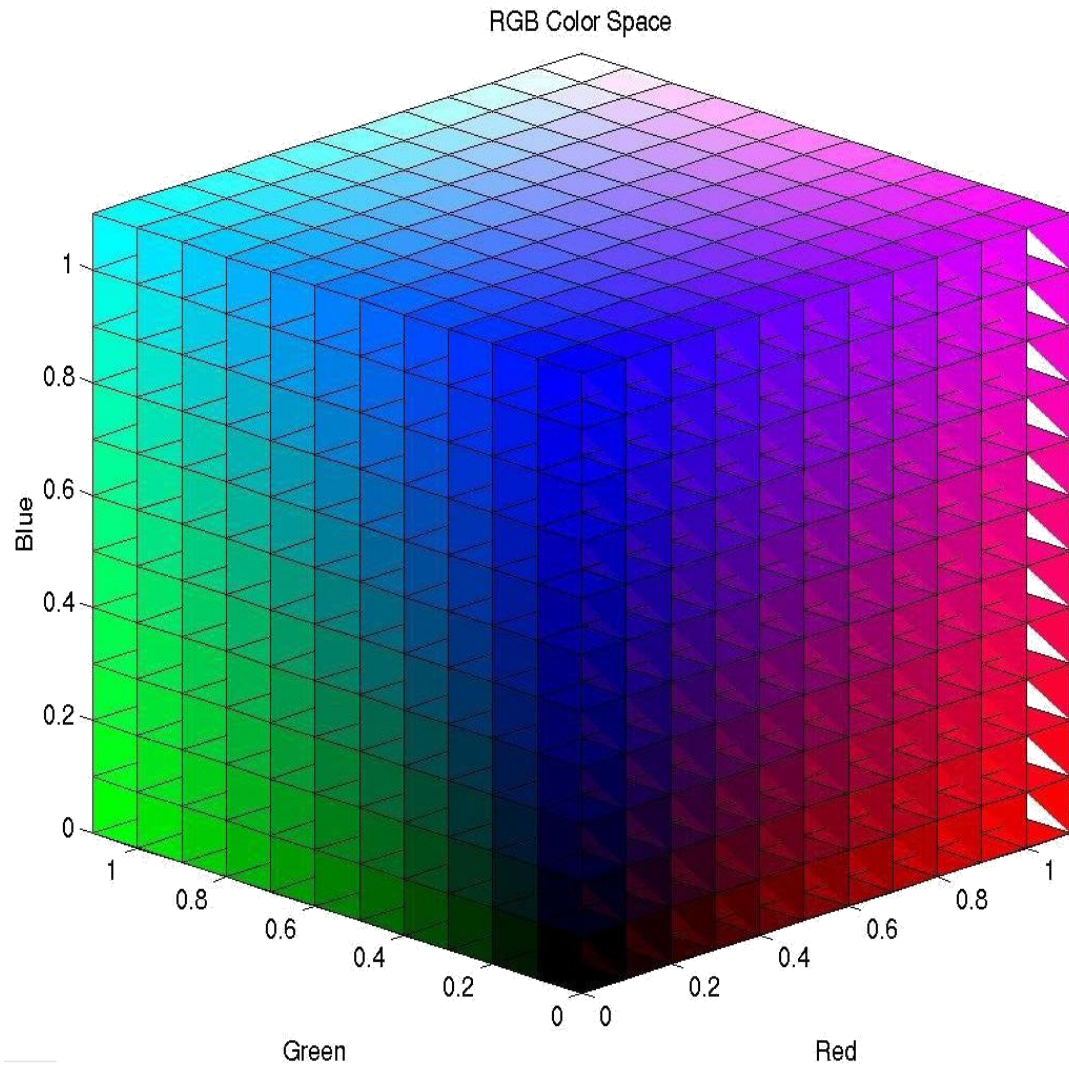




RGB（红绿蓝）是依据人眼识别的颜色定义出的空间，可表示大部分颜色。

在科学研究一般不采用RGB颜色空间，因为它的细节难以进行数字化的调整。它将**色调**，**亮度**，**饱和度**三个量放在一起表示，很难分开。它是最通用的面向硬件的彩色模型。该模型用于彩色监视器和一大类彩色视频摄像。

所以，RGB 颜色空间适合于显示系统，却并不适合于图像处理。





□ 在图像处理中使用较多的是 HSV 颜色空间，它比 RGB 更接近人们对彩色的感知经验。非常直观地表达颜色的色调、鲜艳程度和明暗程度，方便进行颜色的对比。

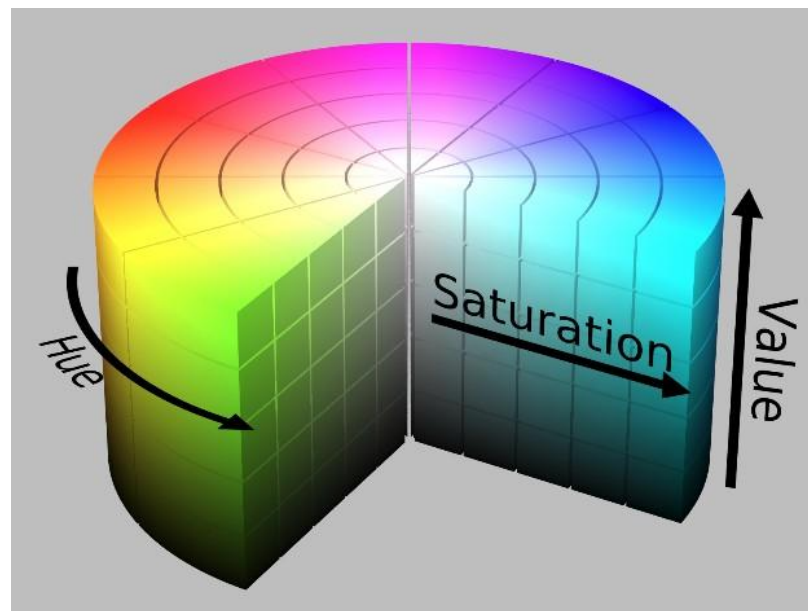
□ 在 HSV 颜色空间下，比 BGR 更容易跟踪某种颜色的物体，常用于分割指定颜色的物体。

HSV 表达彩色图像的方式由三个部分组成：

- **Hue**（色调、色相）、**Saturation**（饱和度、色彩纯净度）、**Value**（明度）。

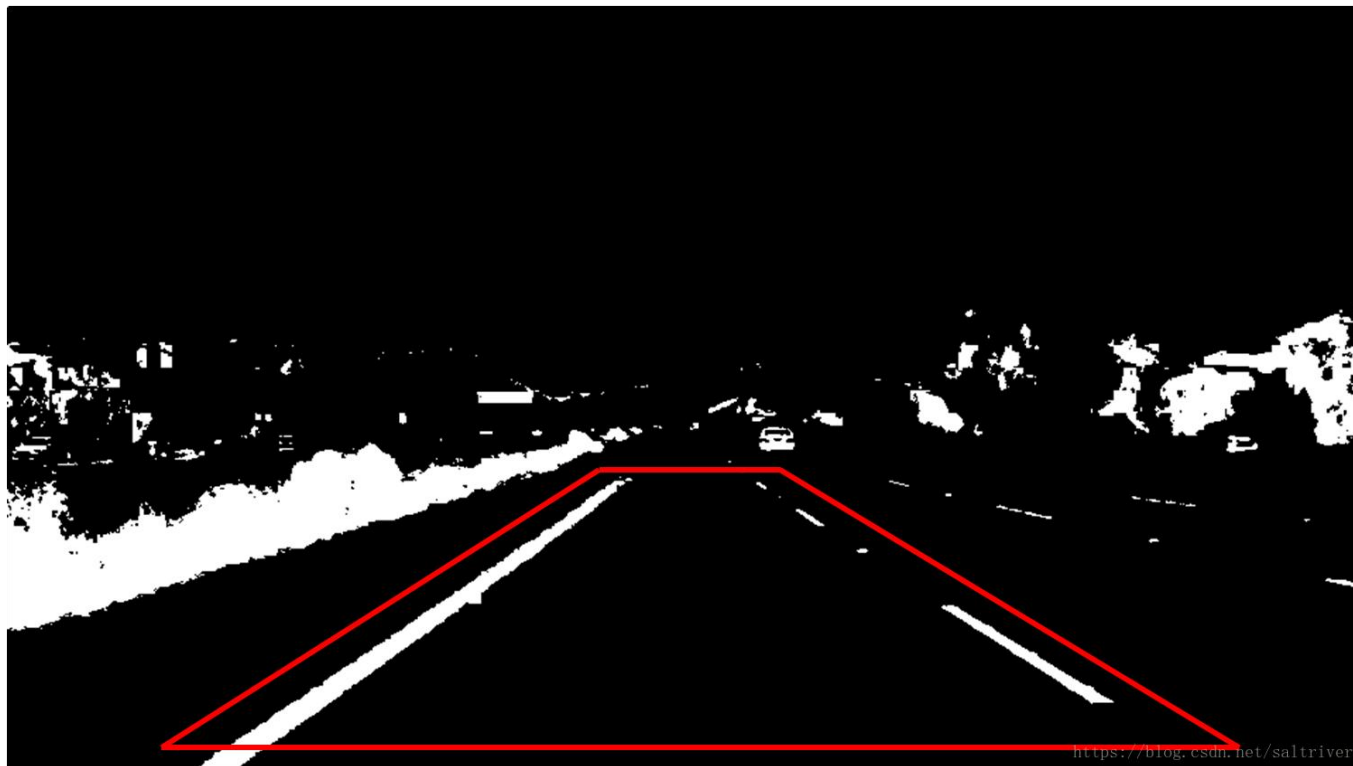
□ 在OpenCV中可用cv2.cvtColor将RGB转为HSV空间，并提取所需的颜色蒙版：

```
# 将图像从RGB空间转换为HSV空间表示
img_hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
# 颜色蒙版
LOWER_RED = np.array([160, 120, 0])
UPPER_RED = np.array([200, 255, 255])
red_mask = cv2.inRange(img_hsv, LOWER_RED, UPPER_RED)
```





- **RoI** (Region of Interest) 指的是图像中感兴趣的特定区域。
- 通过定义和提取**RoI**，可以集中注意力在感兴趣的区域上，从而减少处理时间和资源，并提高算法的效率和准确性。**RoI**可以是任意形状和尺寸的矩形、圆形、多边形或自定义形状。





路线区域信息的提取

为了确定直线的位置信息，先将原始图像从**RGB**颜色空间转换到**HSV**颜色空间，根据赛道线的颜色选择合适的**HSV**阈值范围。得到仅包含赛道线的二值图像，对白色像素的位置进行统计得到其相对于图像中心的偏差，控制小车减少偏差量，使其保持在赛道线上



在HSV颜色空间中
对白色区域
进行提取



截取有用区
域，去除环
境的干扰





霍夫圆检测 是一种基于 **霍夫变换** 的图像算法，用于在图像中检测圆形轮廓。

它的核心思想是：

□ **边缘检测**：先使用 Canny 边缘检测（或其他方法）找出图像中的边缘。

□ **参数空间投票**：圆的数学表达式是 $(x - a)^2 + (y - b)^2 = r^2$ ，其中 (a, b) 是圆心， r 是半径。霍夫变换会在参数空间 (a, b, r) 中投票，找出最可能的圆。



```
circles = cv2.HoughCircles(  
    gray,                                # 输入灰度图像  
    cv2.HOUGH_GRADIENT,                  # 检测方法  
    dp=1,                                # 累加器分辨率（1=与原图相同）  
    minDist=30,                           # 两个圆之间的最小距离  
    param1=50,                             # Canny边缘检测的高阈值  
    param2=30,                             # 投票阈值（越小检测越多，但可能有误  
检)  
    minRadius=10,                         # 最小圆半径  
    maxRadius=100                         # 最大圆半径  
)
```




1

树莓派的使用

2

相机标定与数字图像基础知识

3

基于PID的车辆巡线控制

4

基于卷积网络的标志牌识别

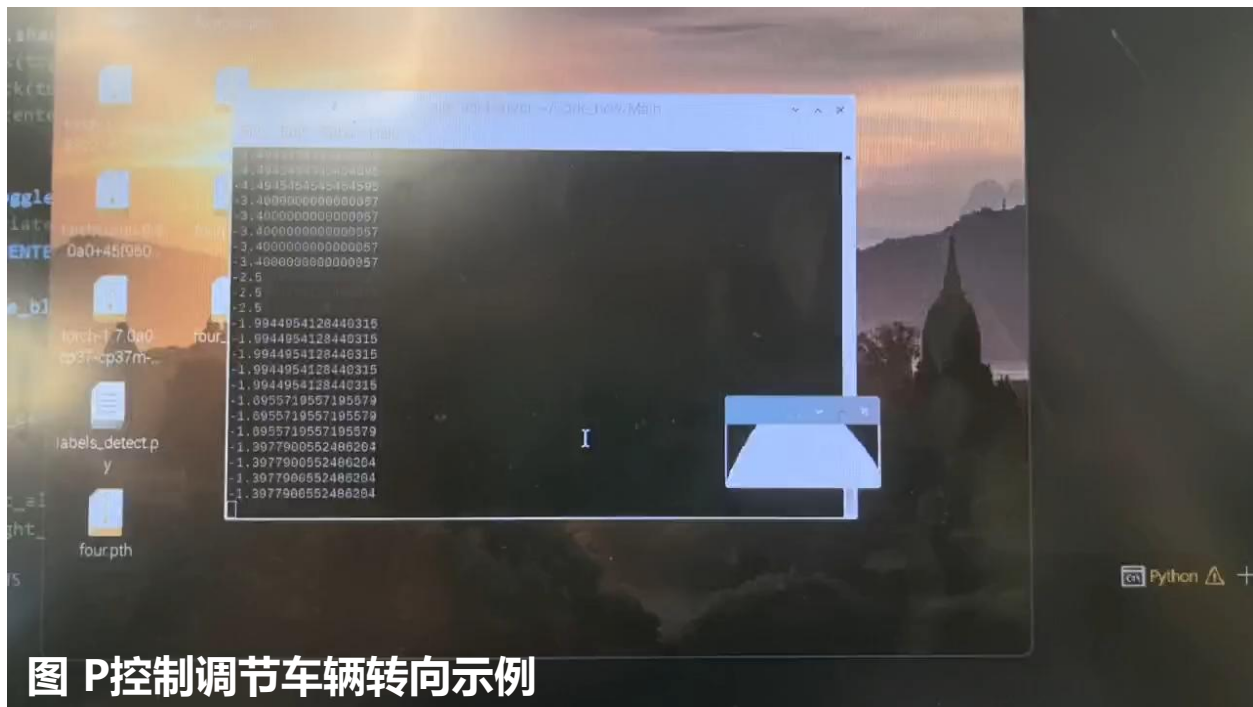
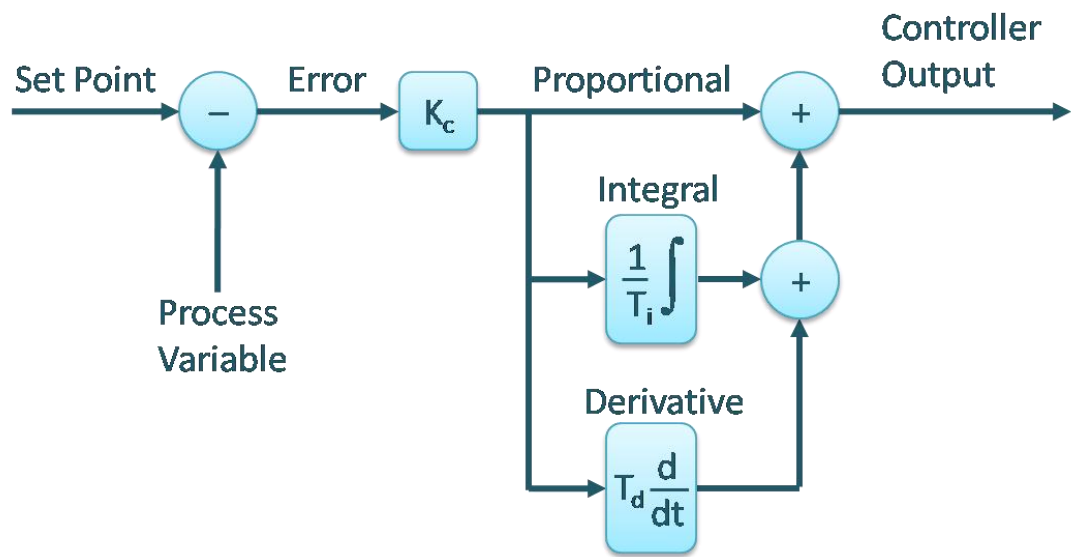


图 P控制调节车辆转向示例



为了使得整个小车的运动过程比较稳定流畅（不会出现小车左右摇摆的情况），需要使用PID控制，这是一种常用的反馈控制机制，用于自动调整系统以达到期望的输出。PID的三个参数作用如下：

比例项 (P)： 增加数值可以让小车快速减少误差，但设置过高会导致小车不稳定，突然向右转后又突然向左转，在中线处摇摆不定；

积分项 (I)： 增加数值可以使得小车严格按照中线行驶，不会产生误差。但设置过高会使得小车偏离中线位置，控制不住；

微分项 (D)： 增加数值可以使得小车不会转向太突然，但设置太高会导致小车速度改变太少，不能起到控制的作用；



1

树莓派的使用

2

相机标定与数字图像基础知识

3

基于PID的车辆巡线控制

4

基于卷积网络的标志牌识别



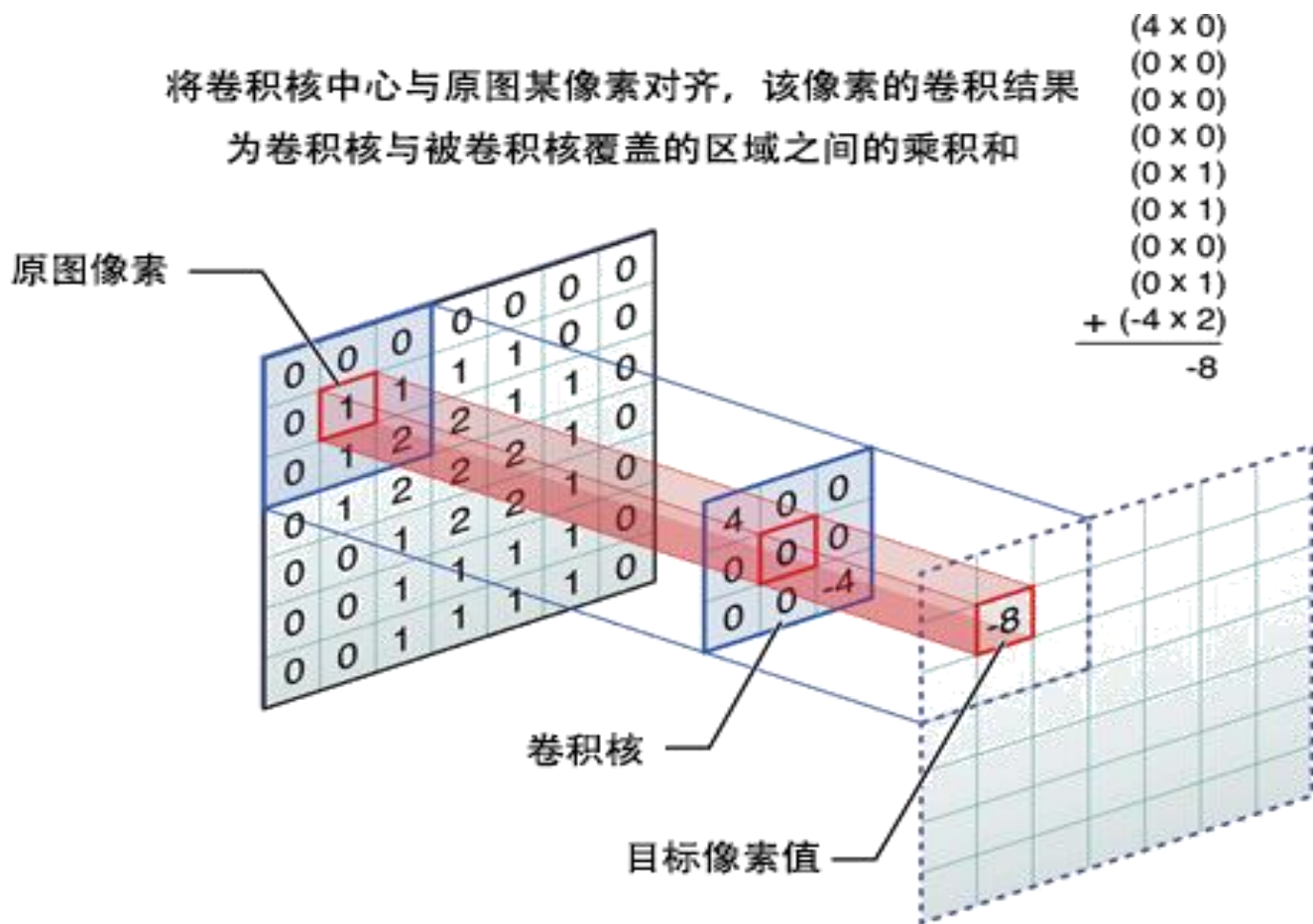
卷积运算 — 基本运算

卷积操作的数学运算过程：

- 左边的是将边界用0填充的原图像，
- 中间是一个 3×3 的卷积核，
- 右边是左上角像素对应的卷积结果。

卷积核滑过整个图像，在每个位置上通过求取卷积核与原图对应区域（被卷积核覆盖的区域）之间的乘积和来得到卷积结果。

卷积步长是每进行一次卷积运算卷积核所移动的像素单位。



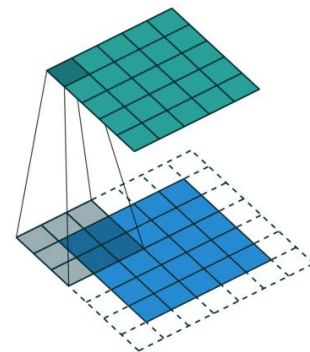


感受野基本概念:

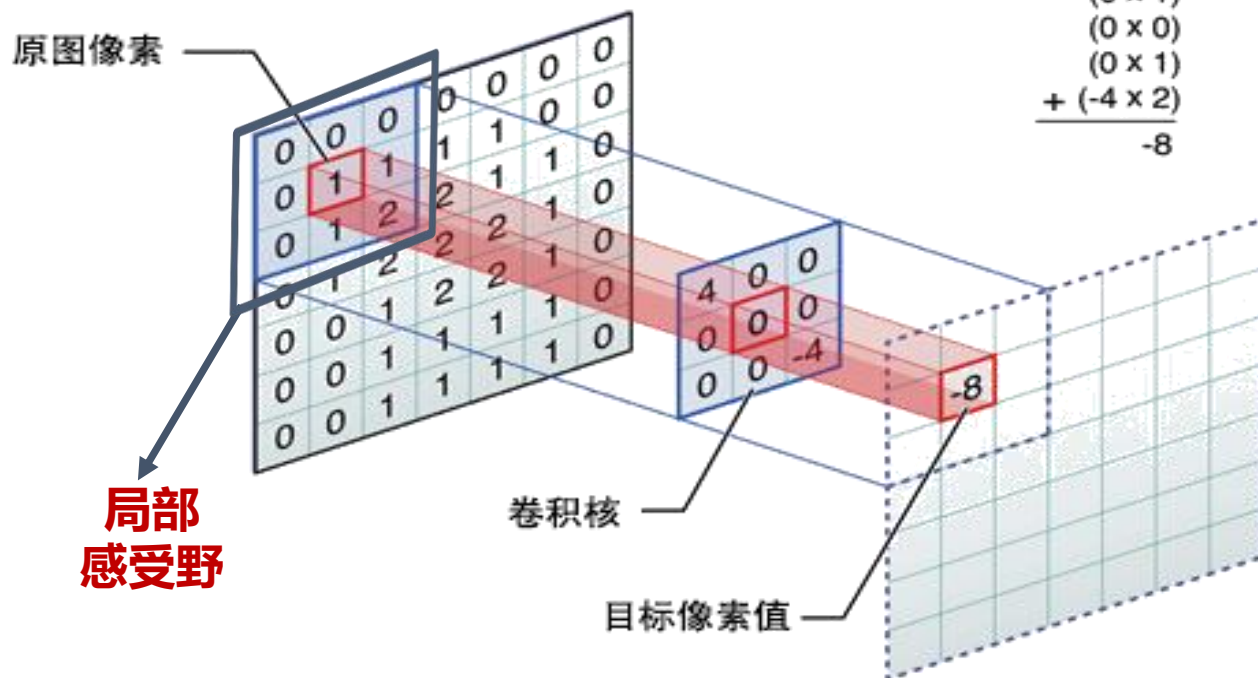
字面意思，感受野是指视觉感受区域的大小。

感受野的定义是卷积神经网络每一层输出的feature map（特征图）上的像素在原始图像上映射的区域大小。

比如上中原图像使用 3×3 的卷积核进行卷积，则图中目标像素值对应的原图像素即为局部感受野。



将卷积核中心与原图某像素对齐，该像素的卷积结果为卷积核与被卷积核覆盖的区域之间的乘积和





卷积核基本概念:

同一个隐藏层（即输出的特征图）中的所有神经元（即输出像素）都是通过同一个卷积核卷积得到，以检测同一个特征在图像的各个位置是否存在，将从输入层到隐藏层的这种映射称为特征映射。

该特征映射的**权重，即卷积核参数**，被称为**共享权值**。

其“共享”的含义源自于：输入图像中的每个局部感受野均被同样的卷积核进行卷积。为了进行图像识别，通常需要不止一个的特征映射，因此一个完整的卷积层包含若干个不同的特征映射，也就是若干个**不同的卷积核**（后面称为“**通道**”）。

卷积核可以改变特征图的尺寸大小。

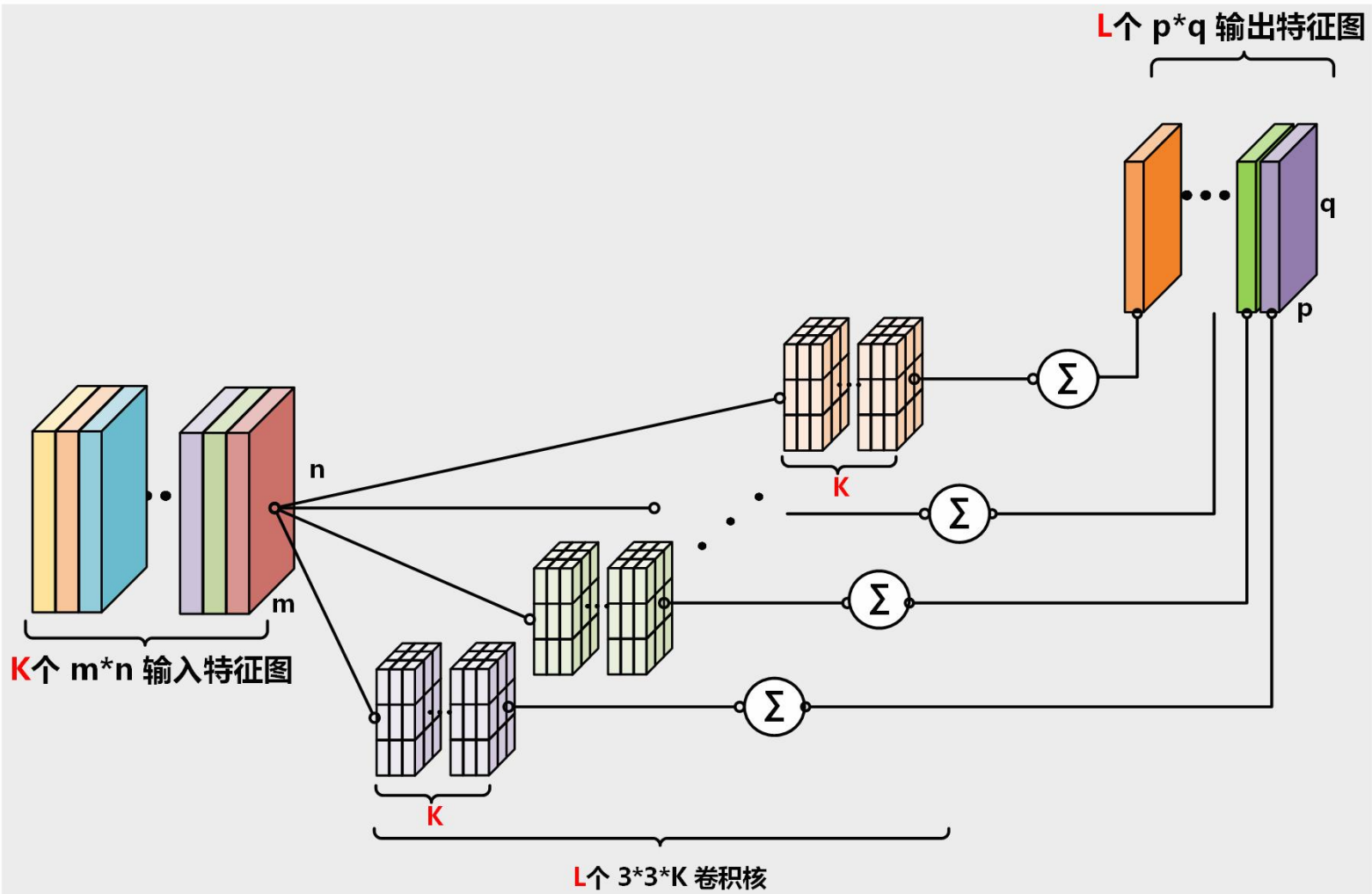
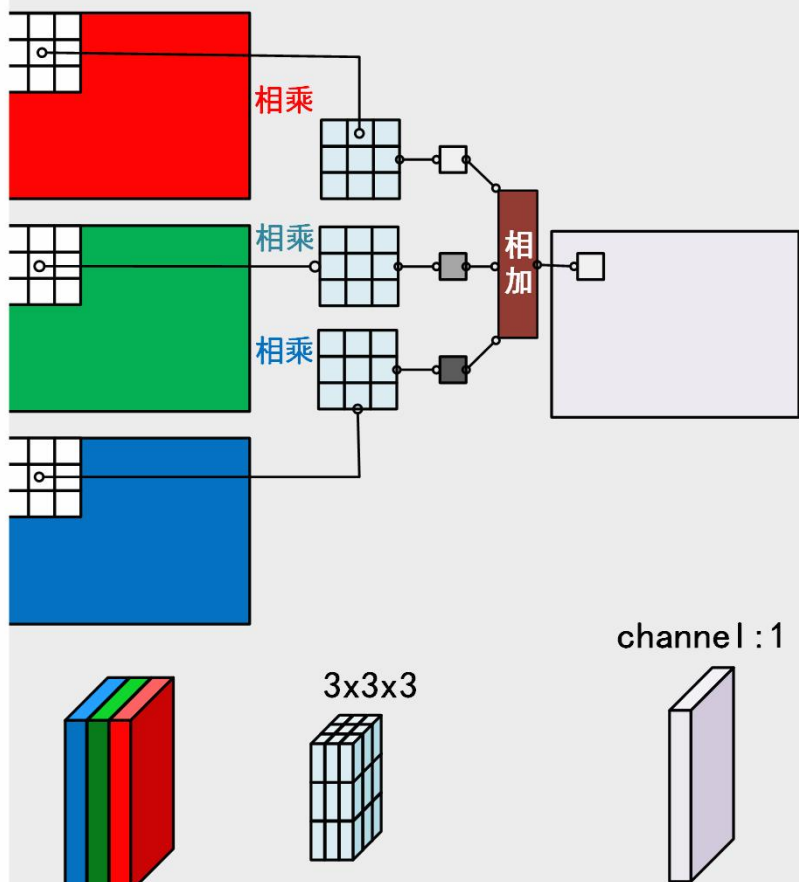


卷积网络 — 基本结构



1组卷积核
 $3 \times 3 \times 3 \times 1$

输出特征图
channel : 1





卷积网络 — 池化层

池化层的主要目的是通过降采样的方式，在不影响图像质量的情况下，**压缩图像、减少参数**。

简单来说，假设特征图像大小为 3×3 ，池化层采用Max Pooling方法，大小为 2×2 ，步长为1，**池化的步长**和卷积相同，也是池化核每两次运算间移动的距离。那么图像的尺寸就会从 3×3 变为 2×2 ，如右图所示。

池化可以调整特征图的尺寸大小

1	-1	2
1	-1	2
1	-2	2

-1	-1	0
1	1	0
1	0	0

卷积后的特征图



1	2
1	2

1	1
1	1

最大池化后的特征图



通常来说，池化方法一般有以下两种：

- Max Pooling，取滑动窗口里最大的值；
- Average Pooling，取滑动窗口内所有值的平均值。

这引发出了一个思考：

从计算方式来看，Max Pooling无疑是比较简单的一种方式，因为只需取最大值即可。那么这样做的意义的意义在哪里？如果只取最大值，那其他的值被舍弃会不会对最终结果有所影响？



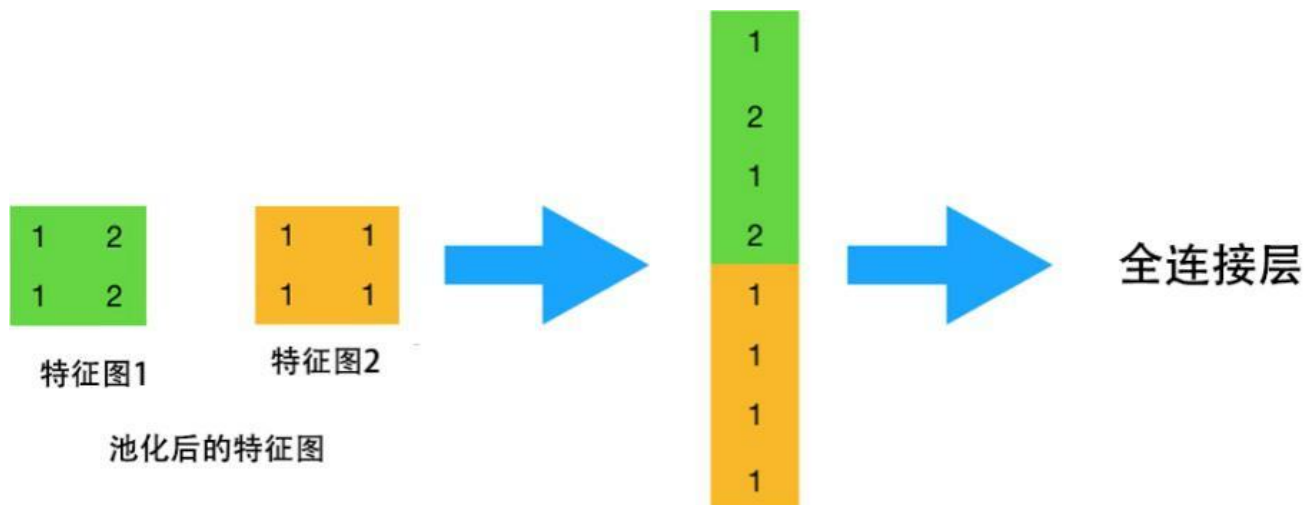
其实每一个卷积核可以看作一个特征提取器，不同的卷积核负责提取不同的特征，假如第一个卷积核能够提取出“垂直”方向的特征，第二个卷积核能够提取出“水平”方向的特征，那么池化对其进行Max Pooling池化操作后，提取出的是真正能够识别到特征的数值，其余被舍弃的数值，这些数值对于提取特定的特征并没有特别大的帮助。但在进行后续计算中，减小了特征图的尺寸，从而减少了参数，达到减小计算量却不损失效果的目的。

池化可以调整特征图的尺寸大小



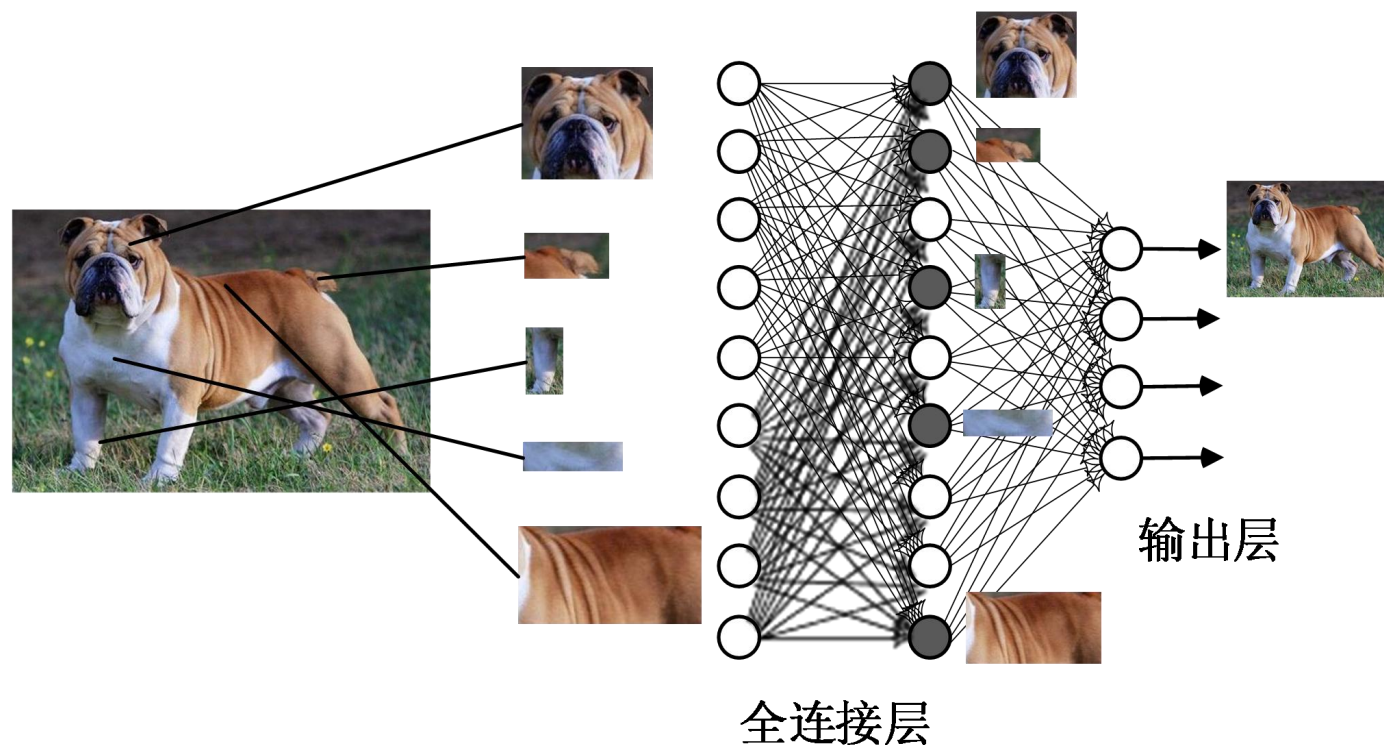
到全连接层 (fully-connected) 这一步，其实一个完整的“卷积部分”就算完成了，如果想要叠加层数，一般也是叠加“Conv-MaxPooling”组合层，因为叠加卷积层可以通过提取更多更抽象的特征来达到更好的检测效果。

在进入到全连接层之前，需要把特征图展开成一维向量，输出到Flatten层（拉伸成一维向量），然后把Flatten层的输出输入到全连接层里，对其进行分类，如图所示。





Flatten层：将所有Feature map拉伸成一维，可以大大减少特征位置对分类带来的影响。也就是说无论在画面哪个位置提取到特征，在进行全连接以后均可被检测到，增强进行检测任务时的鲁棒性。





AlexNet

在 2012 年，Alex Krizhevsky 发表了AlexNet。该神经网络在LeNet的基础上调整了网络架构并加大了深度。

在当年的ILSVRC大赛上 以明显优势夺得了冠军。

ILSVRC：大规模图像识别挑战赛，是近年来机器视觉领域最受追捧也是最具权威的学术竞赛之一，代表了图像领域的最高水平。

这个比赛是斯坦福大学李飞飞教授等人于2010年创办的图像识别挑战赛，极大地推动计视觉发展。比赛项目涵盖：图像分类、目标定位、目标检测、视频目标检测、场景分类、场景解析。



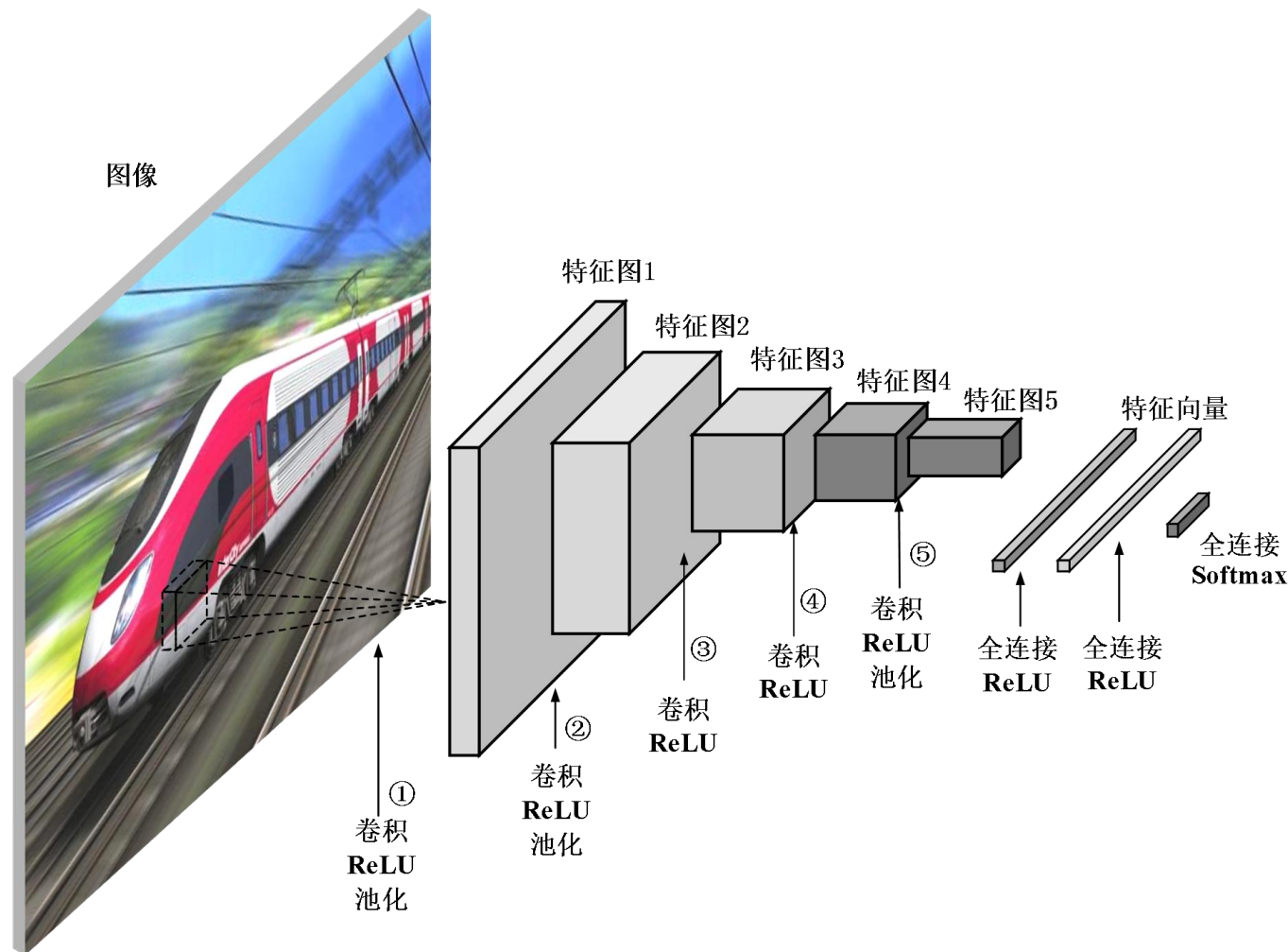
AlexNet

以**AlexNet**作为例子来讲解一个具体的深度神经网络：

- 深度神经网络的结构；
- 相关的基本概念。

这个网络的诞生时间较早，其结构十分基础且**具有代表性**，其中：

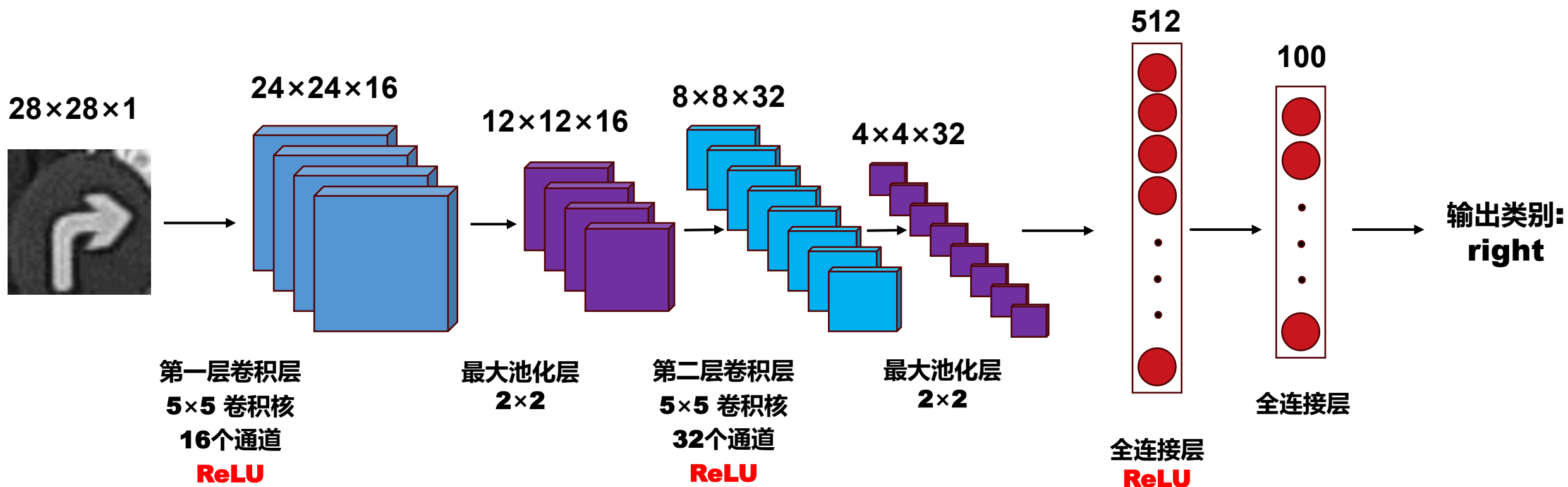
- 卷积层；
- ReLU非线性激活层；
- 池化层；
- softmax归一化指数层；
- 全连接层；



AlexNet 网络结构示意图



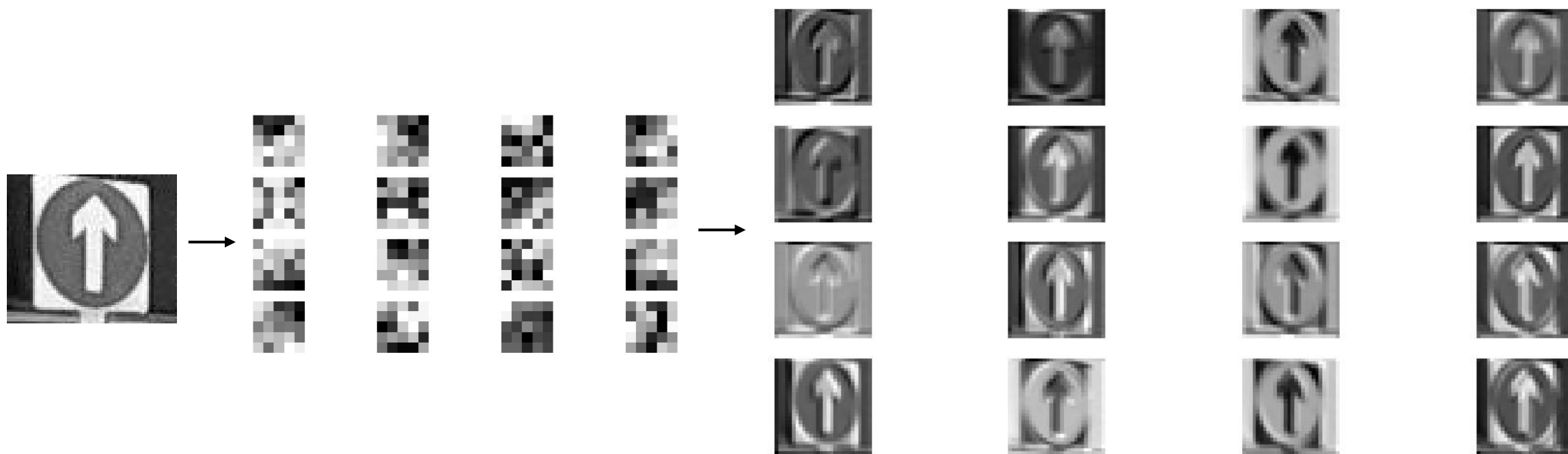
卷积网络 — LeNet网络结构





卷积的可视化

为了更直观地感受卷积，将第一层卷积中的卷积核以及卷积后的特征图可视化如下：



输入图像

16通道的卷积核

特征图：经过卷积操作后的图像



卷积网络的搭建

```
# 定义model
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1),
            nn.Conv2d(16, 16, 5),
            nn.ReLU(),
            nn.MaxPool2d(2, stride=2),
            nn.Dropout(0.3),
            nn.Conv2d(16, 32, 5),
            nn.ReLU(),
            nn.MaxPool2d(2, stride=2),
            nn.Dropout(0.3)
        )
        self.fc = nn.Sequential(
            nn.Linear(32*4*4, 100),
            nn.ReLU(),
            nn.Linear(100, 4)
        )

    def forward(self, x):
        x = self.conv(x)
        x = x.view(-1, 32*4*4)
        x = self.fc(x)
        # x = nn.functional.normalize(x)
        return x
```

使用pytorch中的nn.Module类可以像搭积木一样轻松的实现上述提到的网络:

在init方法中定义网络的组件

在forward方法中定义好数据输入到输出流程



简易标志牌识别模型与推理



```
# 初始化网络 并 加载神经网络参数
net = Net()
net.load_state_dict(torch.load('model.pth',
map_location='cpu'))
net.eval()
# 神经网络预测
with torch.no_grad():
    outputs = net(roi_tensor)
    predict = torch.max(outputs, dim=1)[1].numpy()
    predict_label = int(predict)

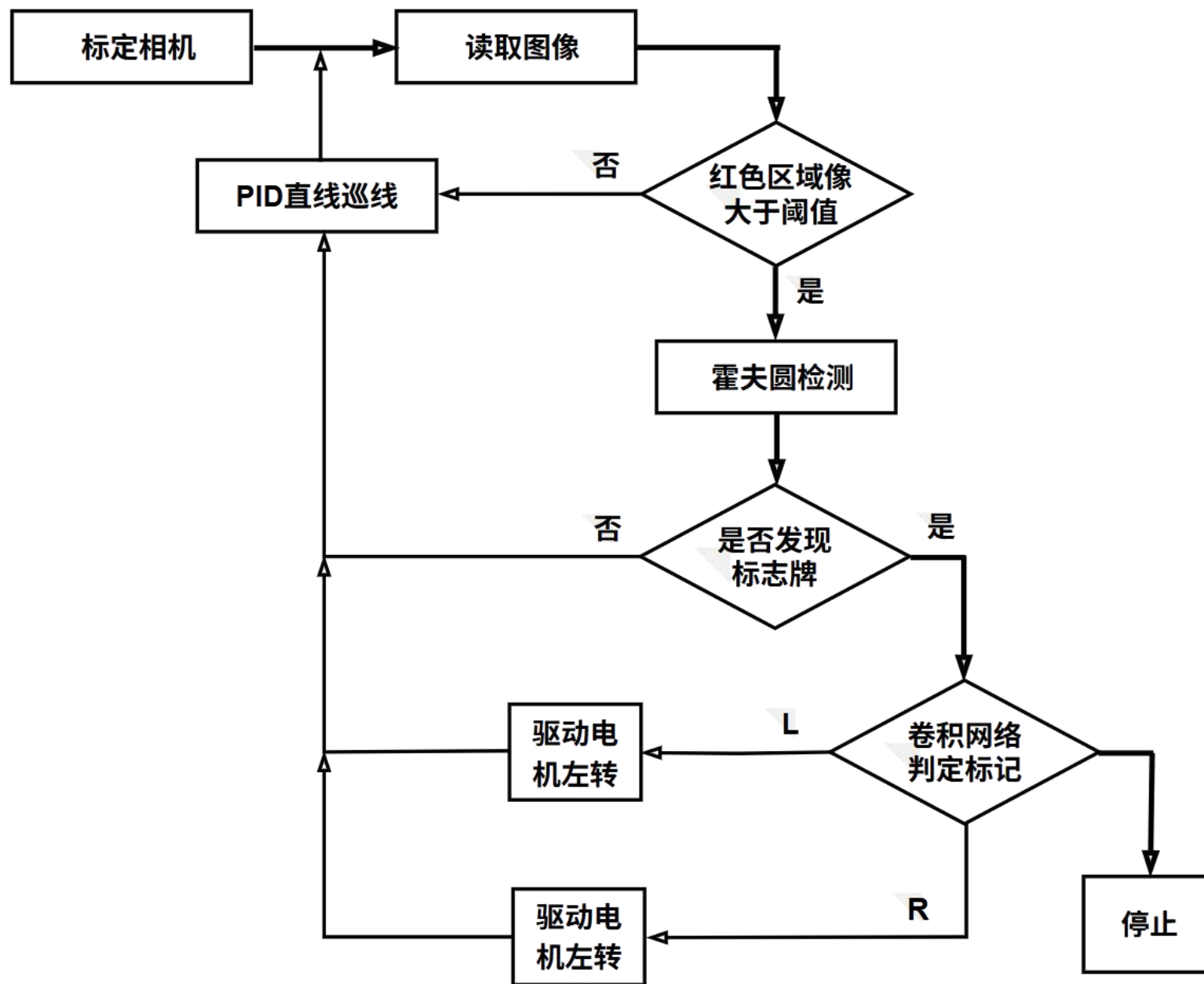
# 设置车辆状态
print("Label: ", predict_label)
if predict_label == 0: # 左转
    carState = State.Left
elif predict_label == 1: # 停止
    carState = State.Stop
elif predict_label == 2: # 右转
    carState = State.Right
```

1. 标志牌识别模型的程序实现见Net.py
2. 在树莓派上部署并加载网络
3. 选取概率最大的类别作为最后识别结果



基于卷积网络的自主行驶技术框架

流程图

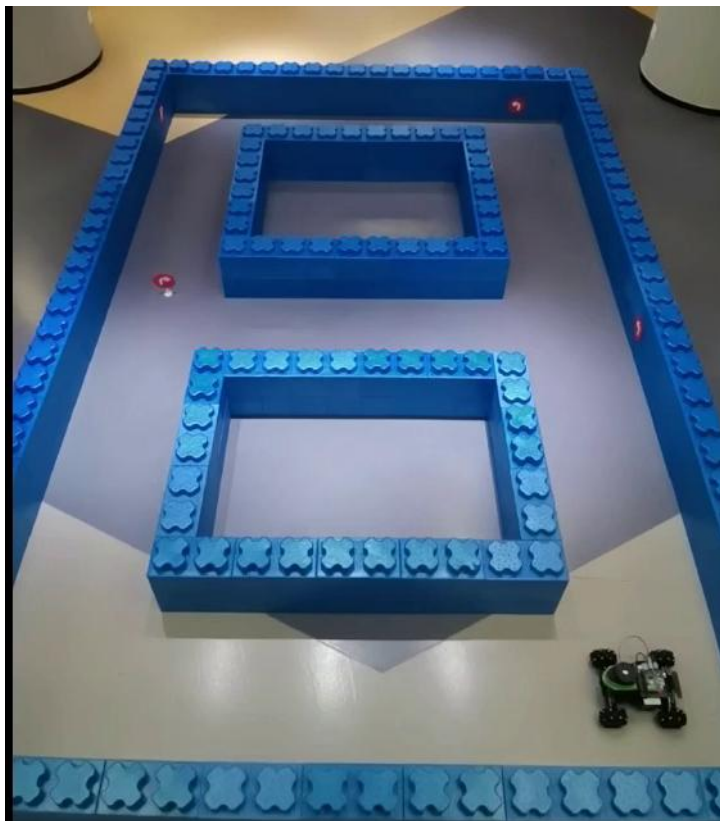




基于卷积网络的标志牌识别

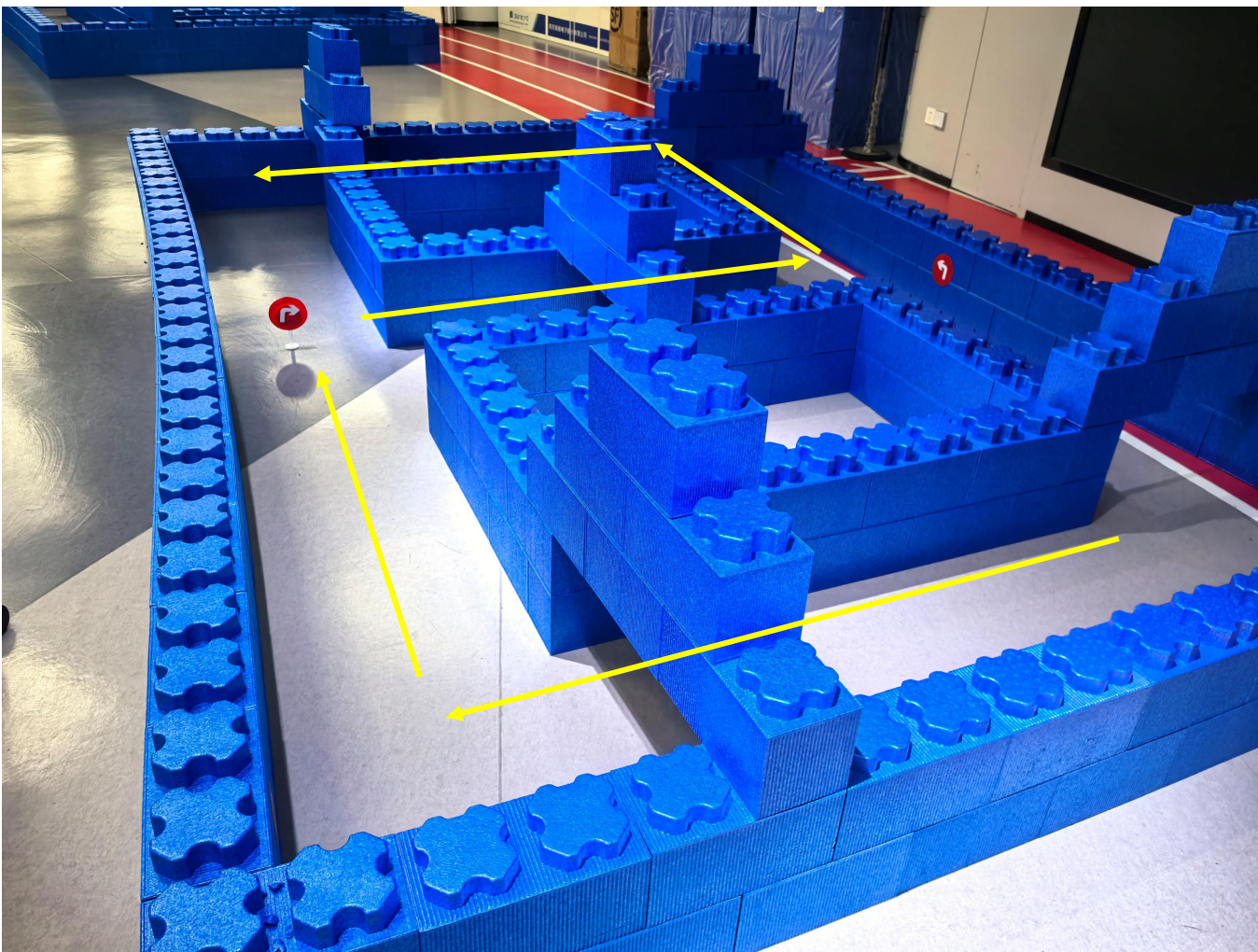


八字形跑道与车载摄像头视角，以及标志牌检测情况





第6部分 视觉与雷达信息融合处理



基于左图8字跑道走S形路线

- 控制小车沿左图中黄色路线行驶,
- 使用车载单目摄像头对路线中的红色标志牌进行识别, 完成相应的转向和停止行动; 若小车能够顺利到达终点, 则视为**任务完成**。

与第5部分的任务要求区别

1. 不用视觉信息巡线
2. 利用雷达信息做行车指导
3. 路口用视觉信息辅助决策
4. 构建视觉与雷达信息融合处理框架